
Badania operacyjne i optymalizacja dyskretna

Sprawozdanie z wykonania zadania

Autorzy: Sebastian Żółkiewicz *259337* Piotr Kulczycki *259257*

Kod przedmiotu: W04ISA-SM0401G, Grupa: 2

Termin zajęć: czwartek 11:15 - 13:00



Spis treści

1	Opis ćwiczenia	2
2	Problem komiwojażera (TSP)	2
3	Algorytmy rozwiązujące	2
3.1	Algorytm Tabu Search (TS)	2
3.1.1	Kluczowe elementy	2
3.2	Przebieg algorytmu	3
3.3	Złożoność obliczeniowa	3
3.4	Przykład działania	3
3.5	Algorytm najbliższego sąsiada (NN)	4
3.6	Algorytm dalekiej insercji (FI)	4
4	Dane wejściowe i reprezentacja rozwiązania	4
4.1	Macierz odległości	4
4.2	Reprezentacja permutacji	4
5	Badania	5
5.1	Metodyka badań	5
5.2	Wyniki	6
6	Wnioski	10
7	Źródła	10

1 Opis ćwiczenia

W ramach ćwiczenia zaimplementowano i zbadano algorytmy tabu search, two opt, NN i FI dla problemu TSP.

2 Problem komiwojażera (TSP)

Problem komiwojażera (ang. Traveling Salesman Problem, TSP) jest jednym z klasycznych problemów optymalizacyjnych w teorii grafów i badaniach operacyjnych. Zadanie polega na znalezieniu najkrótszej możliwej trasy, która odwiedza każde z n miast dokładnie raz i powraca do miasta początkowego. Formalnie można go przedstawić w postaci:

- Dane: Graf pełny $G = (V, E)$, gdzie V jest zbiorem wierzchołków (miast), a E zbiorem krawędzi o wagach d_{ij} reprezentujących odległości między miastami i i j .
- Cel: Znaleźć permutację P wierzchołków, która minimalizuje funkcję celu:

$$\text{Min} \sum_{i=1}^n d_{P(i), P(i+1)} + d_{P(n), P(1)}.$$

TSP jest problemem NP-trudnym, co oznacza, że nie istnieje znany algorytm rozwiązujący go dokładnie w czasie wielomianowym.

3 Algorytmy rozwiązujące

3.1 Algorytm Tabu Search (TS)

Tabu Search (TS) to metaheurystyczna metoda optymalizacji, która znajduje rozwiązania w dużych przestrzeniach poszukiwań. Główne elementy algorytmu to:

3.1.1 Kluczowe elementy

- **Rozwiązanie początkowe:** Algorytm rozpoczyna się od losowego lub heurystycznego rozwiązania.
- **Sąsiedztwo rozwiązania:** W każdej iteracji generowane są nowe rozwiązania w otoczeniu aktualnego rozwiązania.
- **Lista tabu:** Pamięć krótkoterminowa przechowująca ostatnie ruchy lub cechy rozwiązań, które są oznaczone jako tabu, aby zapobiec powrotowi do nich.
- **Funkcja celu:** Ocenia jakość każdego rozwiązania.
- **Kryterium stopu:** Algorytm kończy działanie po osiągnięciu określonej liczby iteracji, czasu lub braku poprawy.

Przebieg algorytmu

1. **Inicjalizacja:** Wybierz rozwiązanie początkowe i utwórz pustą listę tabu.
2. **Iteracje:**
 - (a) Wygeneruj sąsiedztwo obecnego rozwiązania.

-
- (b) Wybierz najlepsze rozwiązanie, które nie jest na liście tabu (chyba że spełnia kryterium aspiracji).
 - (c) Zaktualizuj listę tabu (dodaj nowy ruch i usuń najstarszy).
 - (d) Zaktualizuj najlepsze znalezione rozwiązanie.

3. **Stop:** Algorytm kończy się, gdy zostanie spełnione kryterium stopu.

Algorytm Tabu Search jest szczególnie skuteczny w problemach kombinatorycznych, takich jak problem komiwojażera (TSP) czy harmonogramowanie.

Algorytm 2-Opt

2-Opt to algorytm lokalnej optymalizacji, który poprawia rozwiązania problemów tras (np. problem komiwojażera, TSP). Jego celem jest usunięcie krzyżujących się krawędzi w trasie, co zazwyczaj prowadzi do jej skrócenia.

Założenia

- Rozwiązanie początkowe to pełna trasa odwiedzająca wszystkie punkty w pewnej kolejności.
- Algorytm modyfikuje trasę, zamieniając dwie krawędzie.

3.2 Przebieg algorytmu

1. **Rozwiązanie początkowe:** Wybierz losową lub heurystyczną trasę.
2. **Iteracyjne przeszukiwanie sąsiedztwa:**
 - (a) Wybierz dwie krawędzie $A - B$ i $C - D$.
 - (b) Zamień je, tworząc nowe krawędzie $A - C$ i $B - D$.
 - (c) Sprawdź, czy nowa trasa jest krótsza.
3. **Aktualizacja rozwiązania:** Jeśli nowa trasa jest lepsza, zaakceptuj ją.
4. **Stop:** Algorytm kończy się, gdy nie można już poprawić trasy lub osiągnięto limit iteracji.

3.3 Złożoność obliczeniowa

Każda iteracja ma złożoność $O(n^2)$, gdzie n to liczba punktów w trasie.

3.4 Przykład działania

Założmy, że mamy trasę $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F$:

1. Wybierz krawędzie $B - C$ i $D - E$.
2. Zamień je, aby powstały nowe krawędzie $B - D$ i $C - E$.
3. Nowa trasa to $A \rightarrow B \rightarrow D \rightarrow C \rightarrow E \rightarrow F$.

3.5 Algorytm najbliższego sąsiada (NN)

Algorytm najbliższego sąsiada (ang. Nearest Neighbor, NN) jest prostą heurystyką działającą według zasady chciwej. Rozpoczyna w losowo wybranym mieście, a następnie iteracyjnie wybiera najbliższe jeszcze nieodwiedzone miasto. Proces trwa, aż wszystkie miasta zostaną odwiedzone. Choć algorytm jest szybki, często prowadzi do rozwiązań dalekich od optymalnych.

3.6 Algorytm dalekiej insercji (FI)

Algorytm dalekiej insercji (ang. Farthest Insertion, FI) to heurystyka, która iteracyjnie buduje rozwiązanie, zaczynając od najmniejszego możliwego podzbioru wierzchołków. Algorytm działa według następujących kroków:

1. Rozpoczyna od początkowej trasy złożonej z dwóch najbliższych miast.
2. W każdej iteracji wybiera wierzchołek, który znajduje się najdalej od już odwiedzonych wierzchołków w bieżącej trasie.
3. Dodaje ten wierzchołek do trasy, wstawiając go w miejsce minimalizujące wzrost długości trasy.

4 Dane wejściowe i reprezentacja rozwiązania

Dane wejściowe dla problemu komiwojażera (TSP) opisują miasta, ich wzajemne położenie oraz odległości między nimi. Mogą być reprezentowane na różne sposoby w zależności od specyfiki problemu i zastosowanego algorytmu. W tym zadaniu zastosowano:

4.1 Macierz odległości

Macierz odległości jest jedną z najpopularniejszych reprezentacji danych wejściowych. Jest to macierz $D = [d_{ij}]$, gdzie d_{ij} oznacza odległość (lub koszt podróży) między miastami i i j .

- Jeśli miasta są połączone bezpośrednio, $d_{ij} > 0$.
- W przypadku grafów pełnych każda para miast ma przypisaną wartość d_{ij} .
- Dla grafów nieskierowanych $d_{ij} = d_{ji}$, a w przypadku grafów skierowanych wartości te mogą się różnić.

4.2 Reprezentacja permutacji

Rozwiązanie problemu TSP jest często reprezentowane jako permutacja wierzchołków grafu, np. $[v_1, v_2, \dots, v_n]$, która określa kolejność odwiedzania miast. Długość trasy oblicza się wtedy jako suma kosztów krawędzi w tej permutacji:

$$\text{Długość trasy} = \sum_{i=1}^{n-1} d_{v_i, v_{i+1}} + d_{v_n, v_1}.$$

5 Badania

W ramach przeprowadzonych badań dokonano porównania algorytmów: NN, FI i kilku wariantów tabu search i two opt. Przetestowano warianty tabu search i 2-opt startujące z permutacji losowej oraz jako algorytmy poprawy dla algorytmów NN i FI.

5.1 Metodyka badań

- **Dane wejściowe:** Przeprowadzono testy na różnej wielkości instancjach tsp. Dane zostały wygenerowane losowo dla każdego powtórzenia.
- **Kryteria porównawcze:** Wynik funkcji celu znormalizowano według wzoru:

$$\frac{(A - A')}{A'},$$

gdzie:

- A - wynik funkcji celu testowanego algorytmu,
- A' - wynik funkcji celu algorytmu porównawczego.

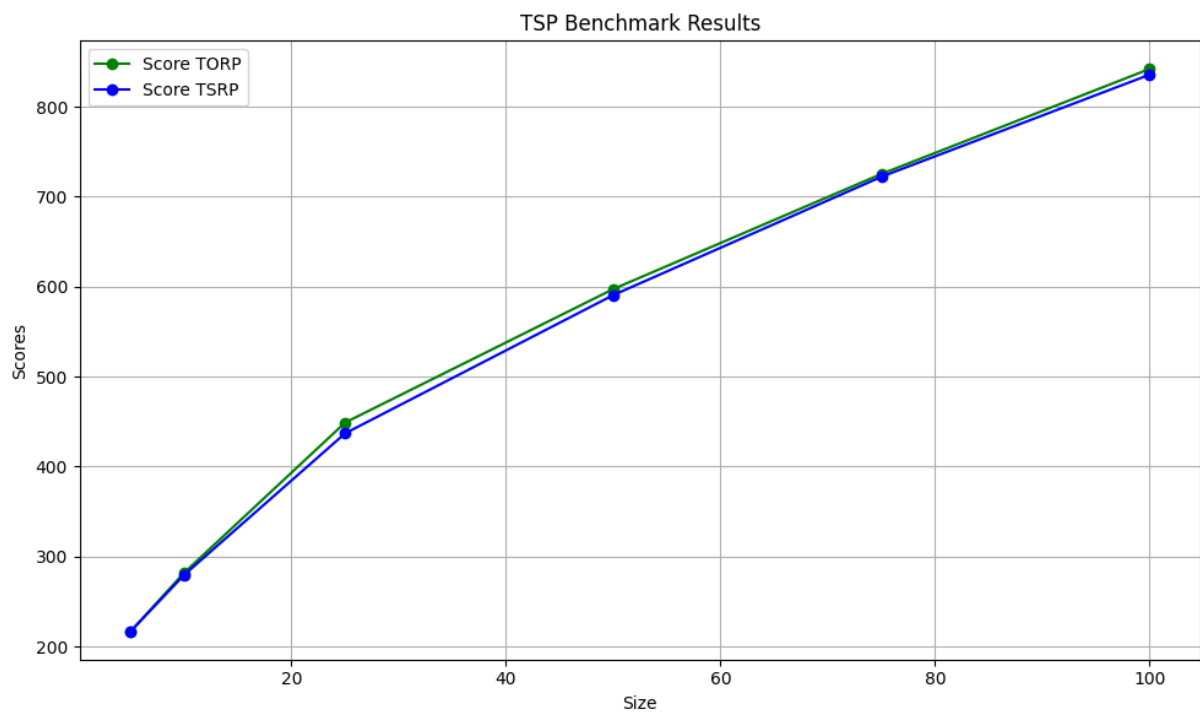
Wyniki uśredniono z 100 prób.

- **Środowisko testowe:** Pomiary przeprowadzono na laptopie MB Pro M1 2021.

5.2 Wyniki

Randpom compare

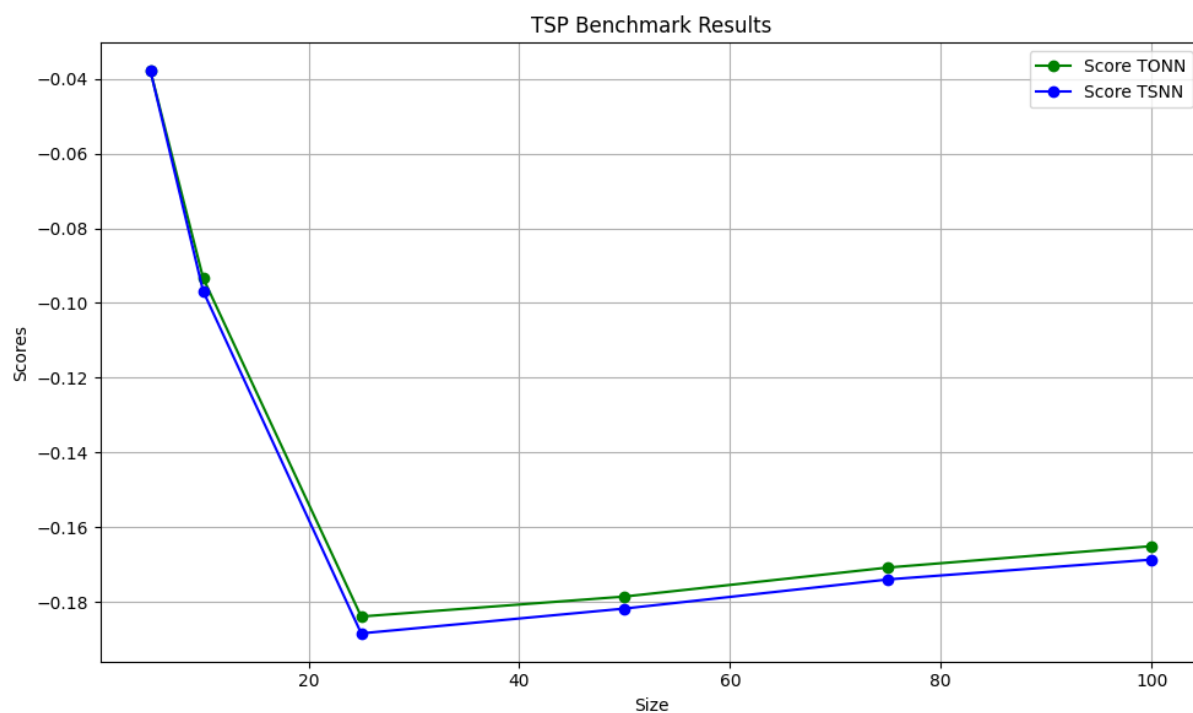
porównanie wyników TO(two opt) i TS(tabu serach) dla losowej instancji początkowej



Rysunek 1: TO vs TS

NN compare

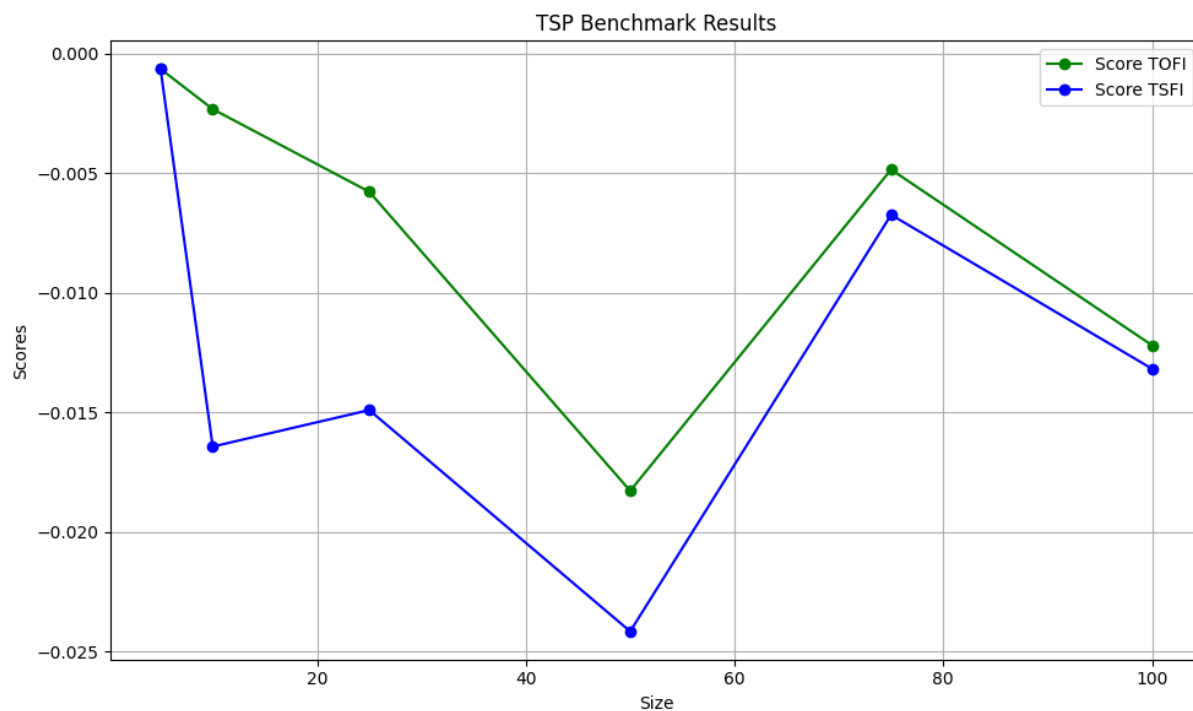
porównanie wyników TO(two opt) i TS(tabu serach) dla instancji początkowej wyznaczonej za pomocą algorytmu NN. Algorytmem porównawczym jest NN.



Rysunek 2: TO vs TS w kontekście NN

FI compare

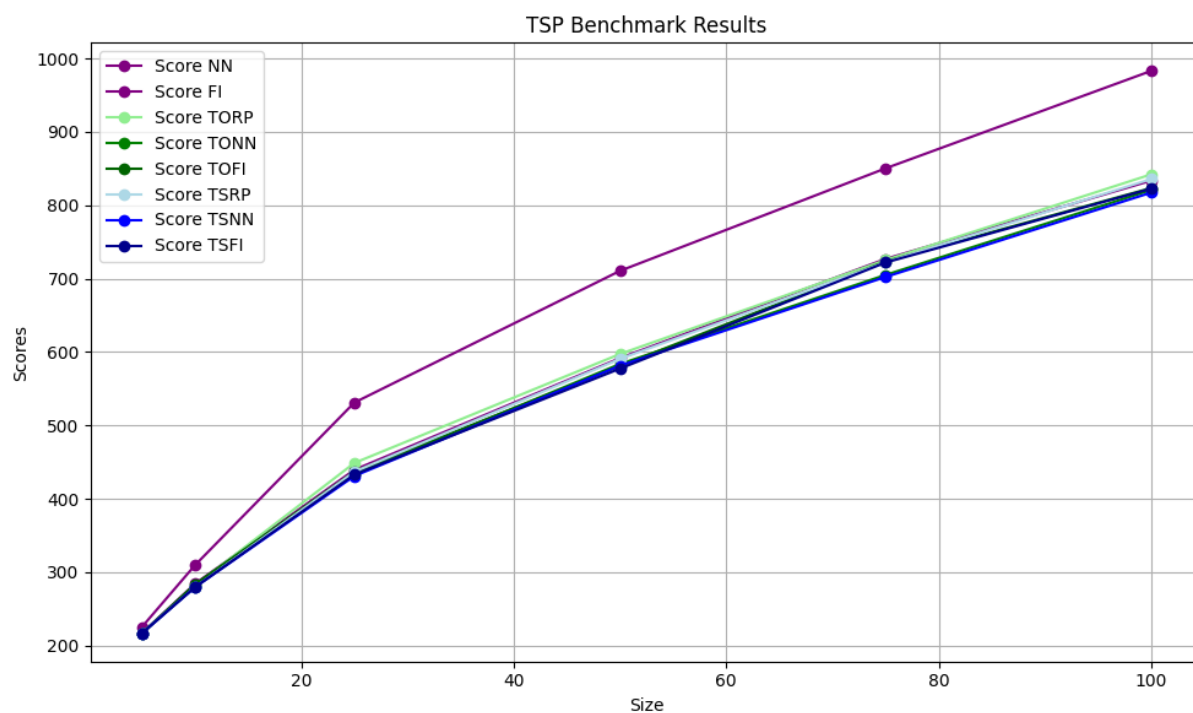
porównanie wyników TO(two opt) i TS(tabu serach) dla instancji początkowej wyznaczonej za pomocą algorytmu FI. Algorytmem porównawczym jest FI.



Rysunek 3: TO vs TS w kontekście FI

Wszystko na raz

porównanie średnich wyników otrzymywanych przez wszystkie alg i kombinacje odpaleń algorytmów poprawiających na konstrukcyjnych



Rysunek 4: ALL

6 Wnioski

- W każdym z badanych przypadków algorytm **Tabu Search** osiągał lepsze wyniki niż **Two Opt**.
- Porównanie algorytmów uruchamianych na instancjach wyznaczonych za pomocą algorytmu **FI** wykazało największe różnice pomiędzy wynikami poszczególnych algorytmów.
- Przy przyjętych algorytmach oraz wartościach hiperparametrów, wyniki obliczeń dla badanych algorytmów okazały się bardzo zbliżone.
- Porównując średnie otrzymane wyniki, najlepsze kombinacje algorytmów prezentują się w następującej kolejności:
 1. **TS** uruchamiany na **NN**,
 2. **TO** uruchamiany na **NN**,
 3. **TS** uruchamiany na **FI**,
 4. **TO** uruchamiany na **FI**.
- Mimo że otrzymane wyniki są zgodne z oczekiwaniami, mogą one silnie zależeć od generatora instancji oraz od charakterystyki samych instancji.
- Najsłabszym algorytmem okazał się **Nearest Neighbor**, który wyraźnie odstaje od pozostałych.

7 Źródła

- Materiały z wykładu i zajęć.
- Materiały dostarczone przez prowadzącego: <http://mariusz.makuchowski.staff.iiar.pwr.wroc.pl>